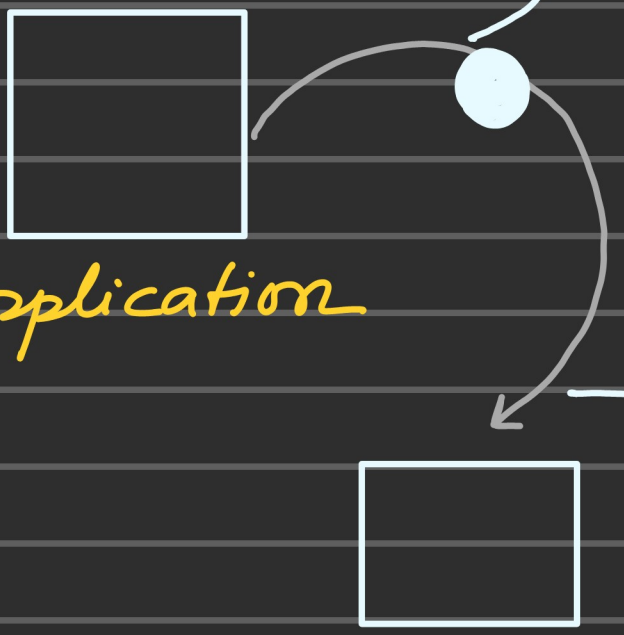
Aws Devops Part - I.

1. Docker:

Packaging tool / software helps to package libraries, dependencies and config files required to run a software.



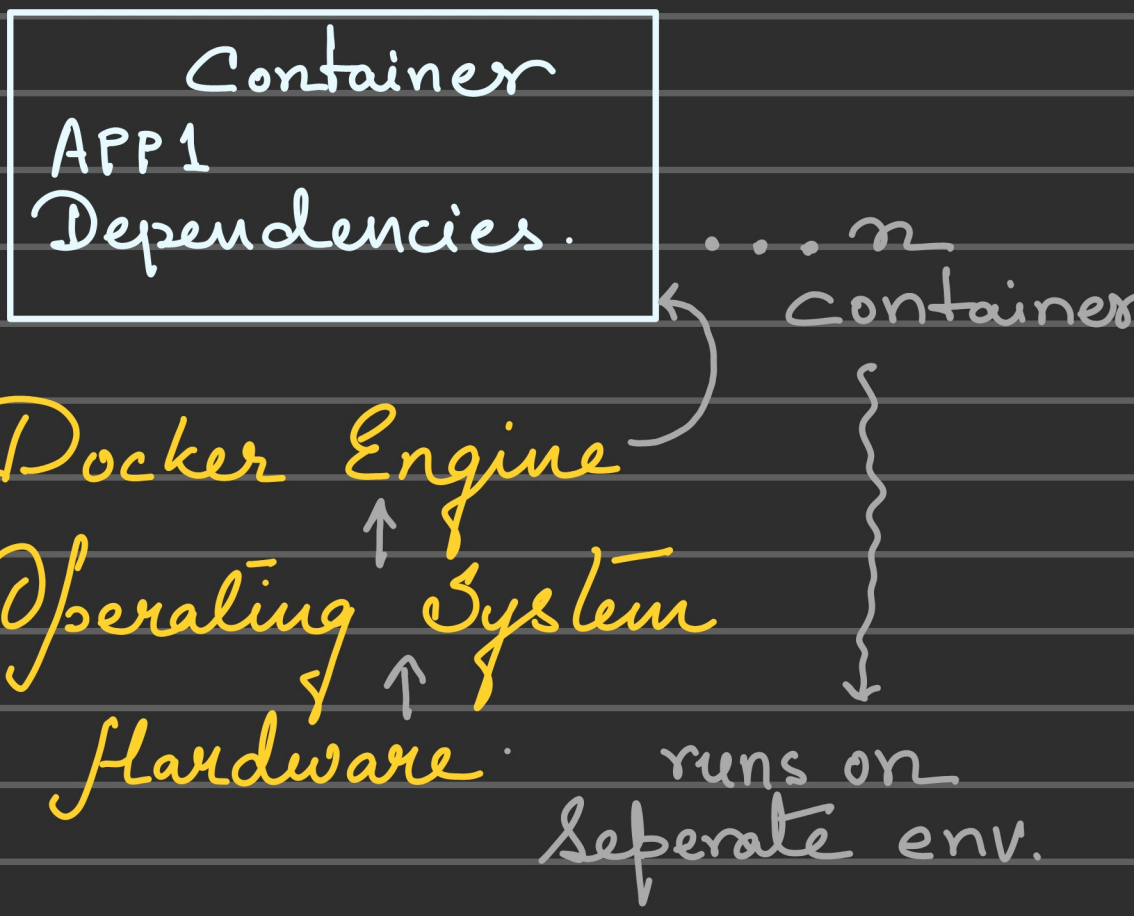
helps to faciliate testing team to run and test software.

I.1.1. Container: A way to package an application with all necessary dependencies and configuration.

Can be easily shared, make development & deployment efficient

Notewise

Architecture of Docker



Docker Container	VMs.
low impact on OS, very fast, low disk usage.	High impact on OS, slower, high disk usage
sharing, re-building and distribution is easy	challenging
Encapsulates apps instead of whole machine	Encapsulate whole machine

Notewise

Docker Engine: the Software that allows us to build run and manage containers on a machine:

Contains:

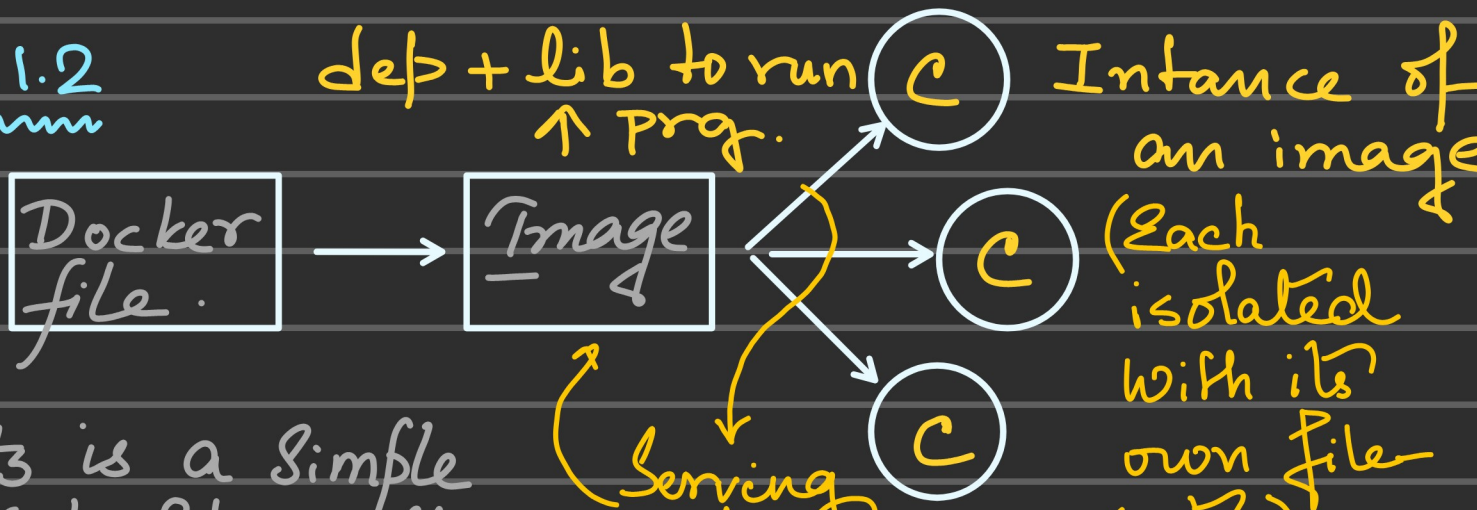
1. Docker Client: Sends cmds to docker eg. docker run, docker build ....

2. REST API: Allow client to communi-cate with daemon... vise versa

3. Docker daemon: background (dockerd) Service that listens for docker API req. and manages docker objects such as containers, images, network .... and Volume

Notewise

I.1.2



Its is a simple txt file with instructions to build an image

I.1.3. Docker registry is simple central repository for storing and distributing docker images.



ECR: Elastic Container Registry; allows to store, share, deploy Docker & OCI artifacts

uses IAM for permissions and scans for vulnerabilities.

I.1.4 Diff. image and Container

Container is the running environment for image.

Notewise

Port → Helps to talk with Appl. running inside container.

Container.
- Fs (virtual)
- Env. Config
- Appl. img.

cmd: docker run -p 8080:3000 myapp
maps 8080 of host to 3000 of Container.

1.1.5 Topics in Overview:

Docker file: text file with instructions to build an image, layer by layer (yml)

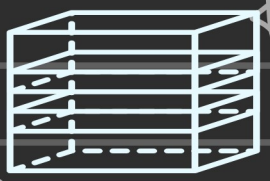
Common Instructions:

- FROM: base image
- WORKDIR: sets working directory inside the image
- CMD: what runs when container starts
- COPY/ADD: copies file from host into image
- RUN: Executes commands at build
- ENTRYPOINT: User argument gets appended
- ENV: sets environment variables.
- EXPOSE: documents which port → container listens on

Notewise

Image: Read-only, immutable template built from a Dockerfile.

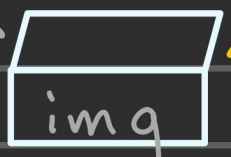
→ Stack of layers + metadata (cmds, ports, env)



(filesystem diffs)

Images are stored locally or in a registry

Container: A running instance of an img.



Docker adds a thin writable layer on top of the image's read only layer

if container is deleted writable layer is gone image remains intact.

any changes during runtime go there

Docker port: Containers have their own private network namespace.

Expose <Port_No.> actually helps to establish connection: host ⟶ container

Notewise

I.1.6 Amazon Elastic Container Service

AWS container orchestration service — runs and manages Docker containers at scale

Task definition: JSON blueprint → describes which images to run, CPU/memory, ports, envs.

Task: a running instance of task definition.

Service → keeps a specified number of task running, handles restarts, integrates with load balancers.

I.1.7. Architecture of Docker Container

(a) Filesystem: inside a docker container is layered, designed for • speed, • efficiency, • portability

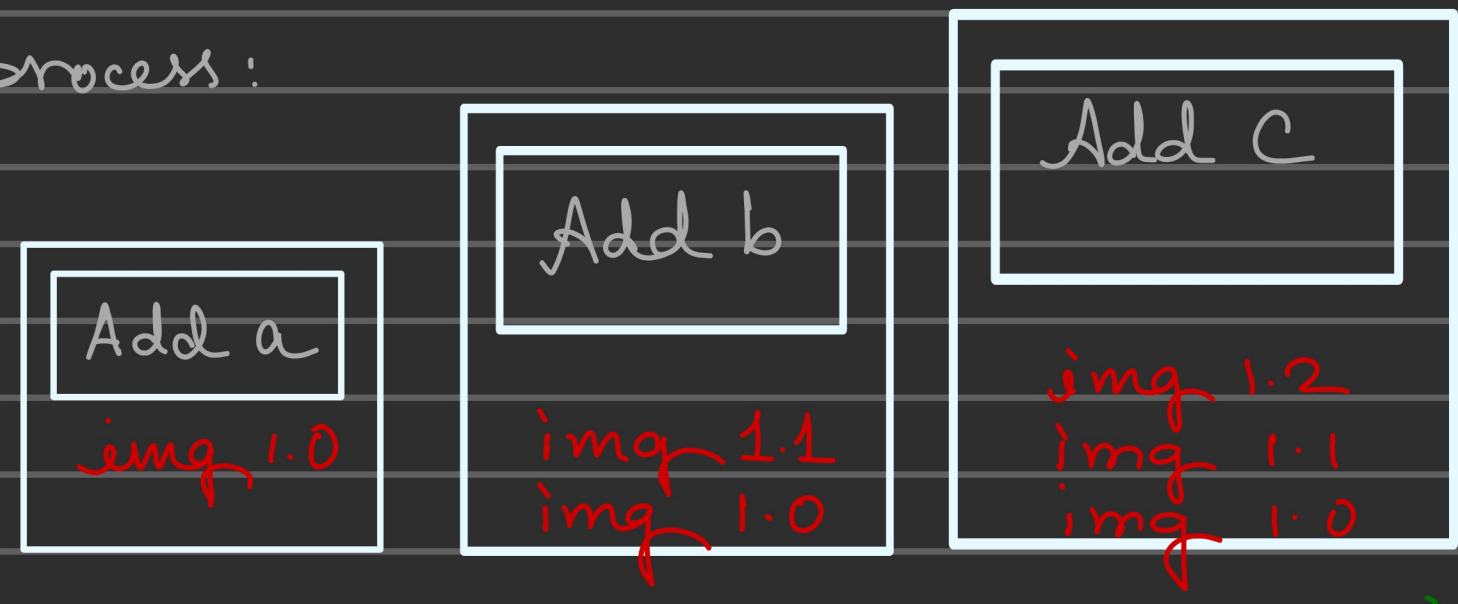
(I) Union filesystem:

→ × Monolithic Disk image

→ ✓ layered Union FS, with stacks of lightweight layers on top of each other

Notewise

process:



Appearing as single unified system.

Image layers (Read-Only):

creates a new read-only layer

shared across any container that uses the same image

Container layer (Read-Write):

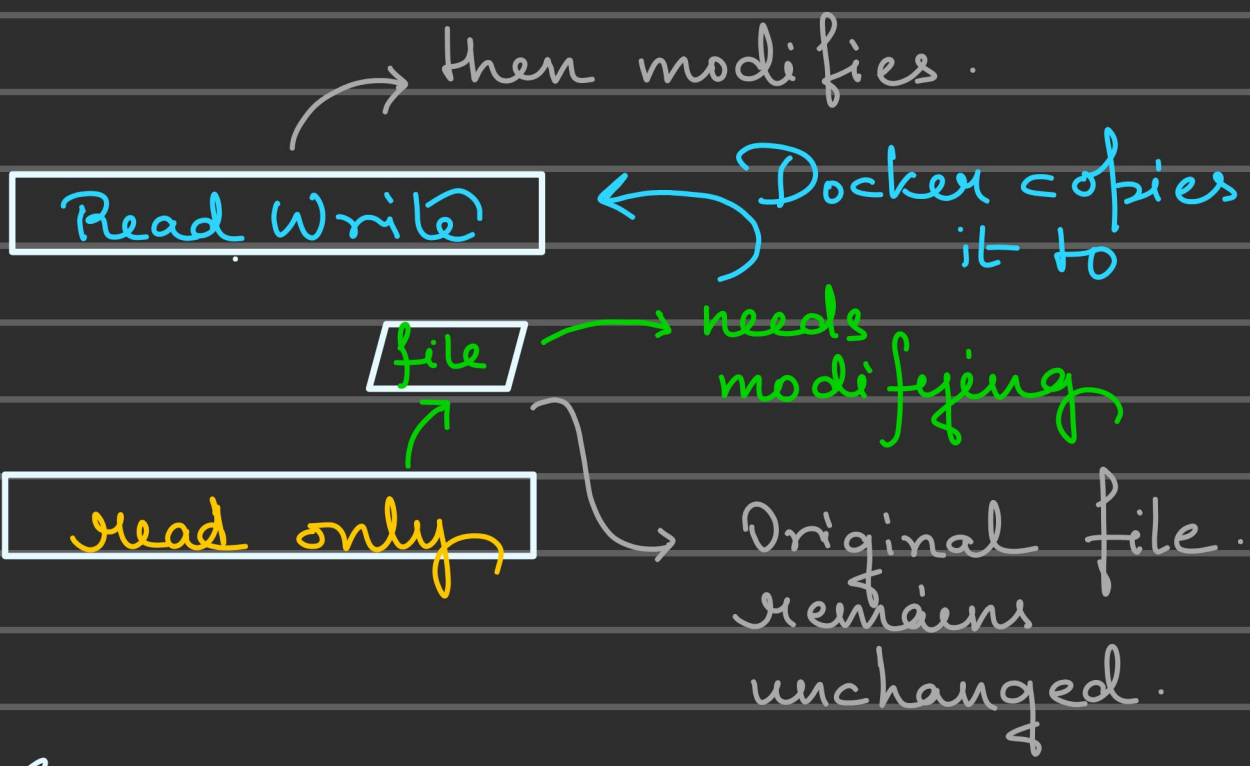
img → container

↳ Docker adds a thin ephemeral layer (rw)

any changes happens in this layer only

Notewise

Copy-on-Write (CoW):



② Storage Drivers:

Required for handling and managing this complex stacking and merging of layers.

Eg. (latest) Overlay2,
(legacy) aufs, btrfs, zfs and devicemapper.

③ Persistent Storage. (Data Management)

facilities provided by docker to preserve data in container layer.

Notewise

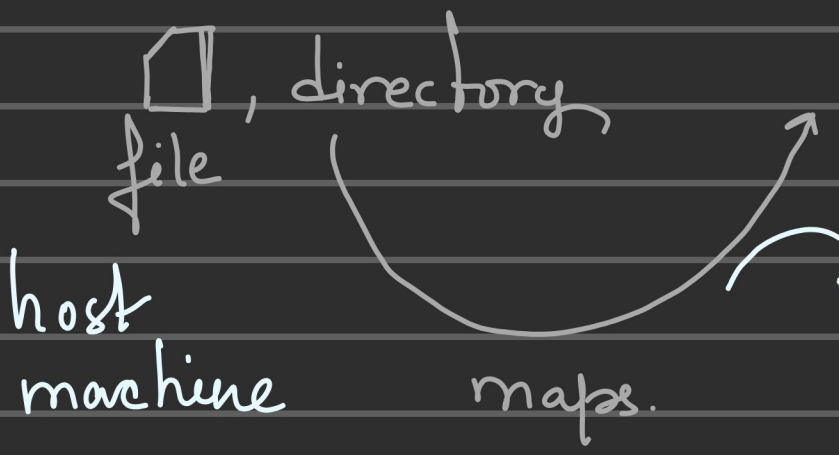
host OS.

Volumes: Safest and preferred way to persist data

managed by docker, isolated from the core functionality of the host machine.

- Easy to backup, share
- Work seamlessly across different OS environment.

Bind Mounts:



So changes reflect instantly without rebuilding

tmpfs Mount: Stores data strictly in host's system memory (RAM)

Notewise

(6) Namespaces: linux kernel feature

↳ Namespace?

fs → dictates what Container can see on the disk.

Namespaces → dictates what container can see regarding OS.

→ provide an illusion → (C)ontainer.

running on its own dedicated machine

isolated System resources from process inside container

→ No interference with process Outside of it.

Isolation

6 core namespaces:

- PID:
- NET:
- MNT
- USR:
- HOSTNAME:
- ?

Notewise

Cgroups.

Cgroups determine what a container can use

Cgroups → limits, accounts, isolates → resource usage.

→ Without it potential problems may arise.

- lets assume a container with a memory leak; CPU-intensive loop could consume 100% of HM. resource

Starving all other containers and potentially crashing machine.

to configure limits on container, config these cgroups limits using flags. in <docker run> or <docker-compose.yml>

Notewise

Resource	Task
CPU	Restrict Container to Specific fraction of CPU cycle/pin to → Specific CPU Core.
memory	hard limit on ram consumption. above this limit linux OOM terminate process
Block I/O.	limits Speed at which Container R/W → disk prevents bandwidth monopolizing
Network.	Shaping and throttling outbound Net traffic to prevent bandwidth exhau..
Device.	Controls access betn Container → physical/Virtual devices on host system.

Noisy Neighbour problem – multitenancy

→ Soln → Cgroups. provides isolation and partioning resources within boundaries.

Notewise

Docker Client & Networking (T.1.8)

Docker Client is the command line tool, used to Communicate with dockerd (Docker daemon)

When we run a command such as 'docker run nginx' DC → DD responsible for creating and managing container.

popular docker commands:

docker pull — Downloads an image

docker run — Creates and starts a container

docker images — lists all images

docker build — Creates an img from docker file.

docker ps / ps -a — shows running, all containers

docker stop, rm, log <id>

docker exec -it <id> bash — interactive mode

Notewise

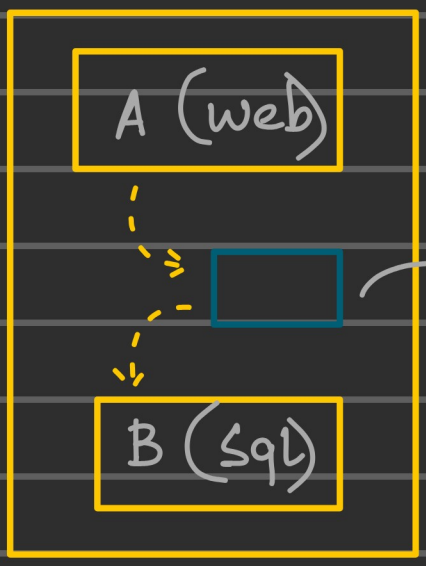
Docker Networking

Allows containers to communicate with:

→ Other Containers
→ Docker host
→ External Networks.

• via ports.

(C→C.)



Step. docker network create mynet

docker run -d --name web --network mynet nginx

mysql req

Web container ---------> db Container

docker run -d --name sql --network mynet mysql.

(Host → Container) done with help of container ports.

port is where appl. in container listens to communicate (browser → Host:port → Docker port fwd → Container port → Appl)

Notewise

C → Outside Network.

curl https://ex.com.

Process: C →
Docker Network
↳ Docker Host
Internet
↳ External Server.

<goes thru Docker NAT on host and reaches Outside Network.>

Ext Server (sees) → Host IP
↳ NAT
containers pvt.
Docker IP.

Outside Net → C.
(user)

Outside user wants to access docker container

docker run -d
--name web -p
8080:80 nginx
Host → C.
P P

Routing would be done like.

User →
(http://hostIP:8080)
↳ Docker host
container:80
↳ Nginx.

Docker Overlay Network

Container to container communication across different host machines

Notewise

docker network create -d overlay my-net.

docker service create --name web --network my-net nginx. C_1 (M1)

docker service create --name db --network my-net mysql. C_2 (M2)

Web Container

↓ http://api:8080

Overlay Network

Encapsulated network traffic

Docker Host 2

↓

API Container : 8080

Swarm Cluster

Container A

↓ Original packet

Docker Host 1

↓ Encapsulates packet

Physical/Cloud

↓

Docker Host 2

↓ Decapsulates packet

Container B

Manager Node

↳ Worker Node 1

↳ Worker Node 2

Notewise

(Extras: I.1.9 Docker Compose)

A tool for running multi-container application with a single yml file instead of running a bunch of separate docker run commands.

(manual)

Eg. docker network create mynet
docker run -d --network mynet --name db -e ..... -v
docker build -t web
docker run -d --network mynet --name web -p ..... -e .....

Compose:

docker compose up. create and start all services
down stop and rm -v (vol)
build rebuild img
logs -f tail logs across service
ps. list running services in this project.

Key Concepts: Each one becomes a container (service)
Compose auto-creates a shared network
attaches named volume for persistent

'depends ON' : 'Wait Until ready' - manages startup order

build vs image : build → local
image → registry

Notewise

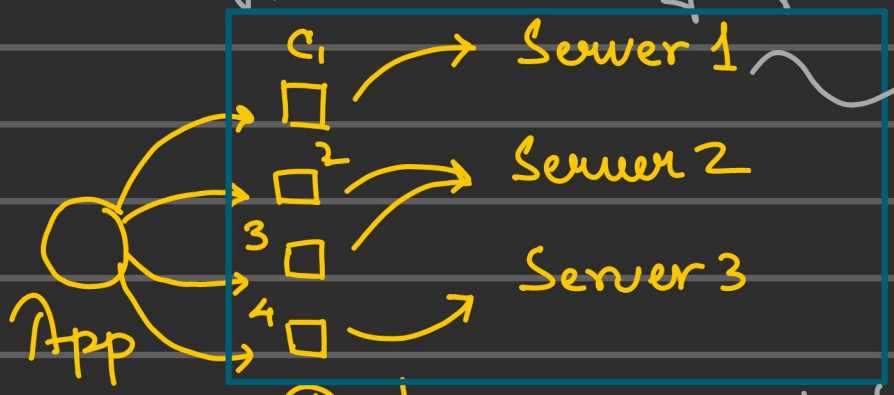
Aws Devops Part - II

2. KUBERNETES

Container orchestration Service.

Kubernetes also known as K8's, Open-source system for.

> Automating deployment
> Management of containerized apps.
> Scaling & Monitoring of Containers



C_3 crashes kubernetes automatically creates new container

can also add C's when traffic is increased (Auto-Scalling)

< load balancing >

Notewise

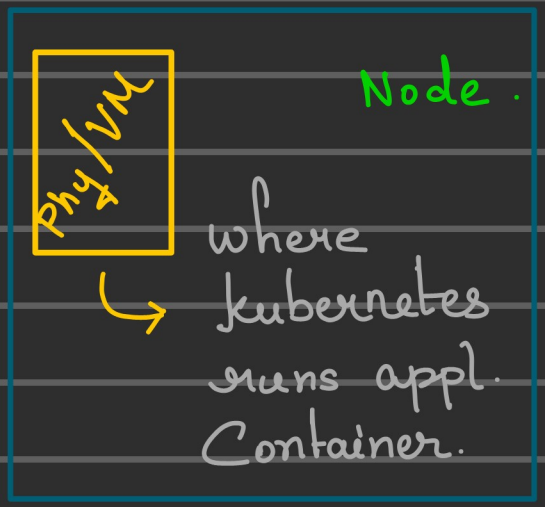
What is exactly Container Orchestration:

Automatic Management of containers which involves:-

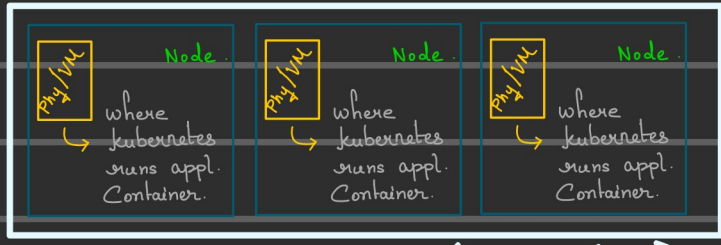
- deploying
- Start / stop.
- Scalling
- Networking
- load-balancing
- Replacing crashed Cn

II.2.1 Server (Nodes): A Server (Node) in kubernetes (physical/virtual) machine in a kubernetes Cluster where Containers run.

II.2.2 Architecture of Kubernetes:



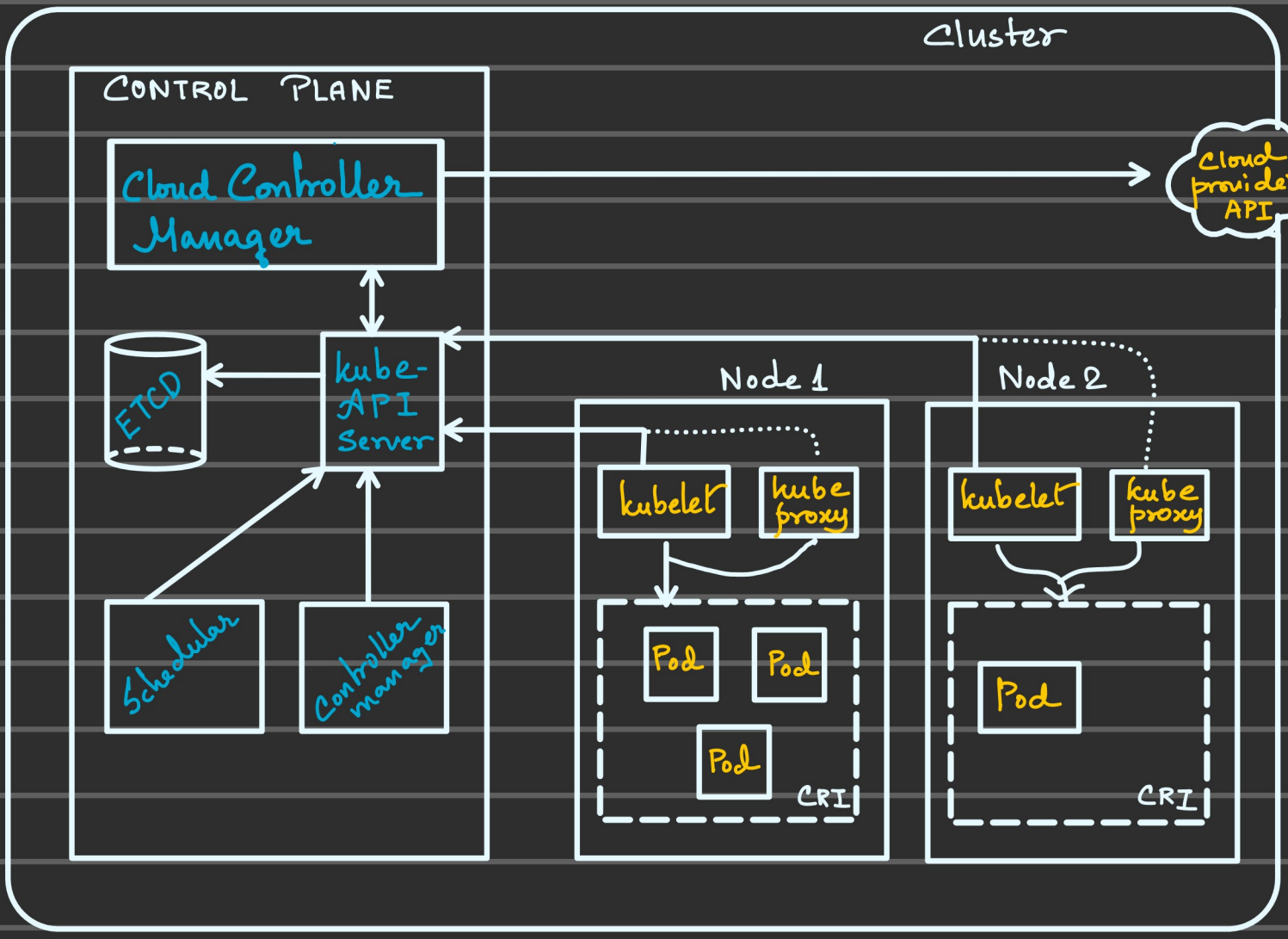
→ A kubernetes cluster consists of group of nodes (1 or more)



(Cluster)

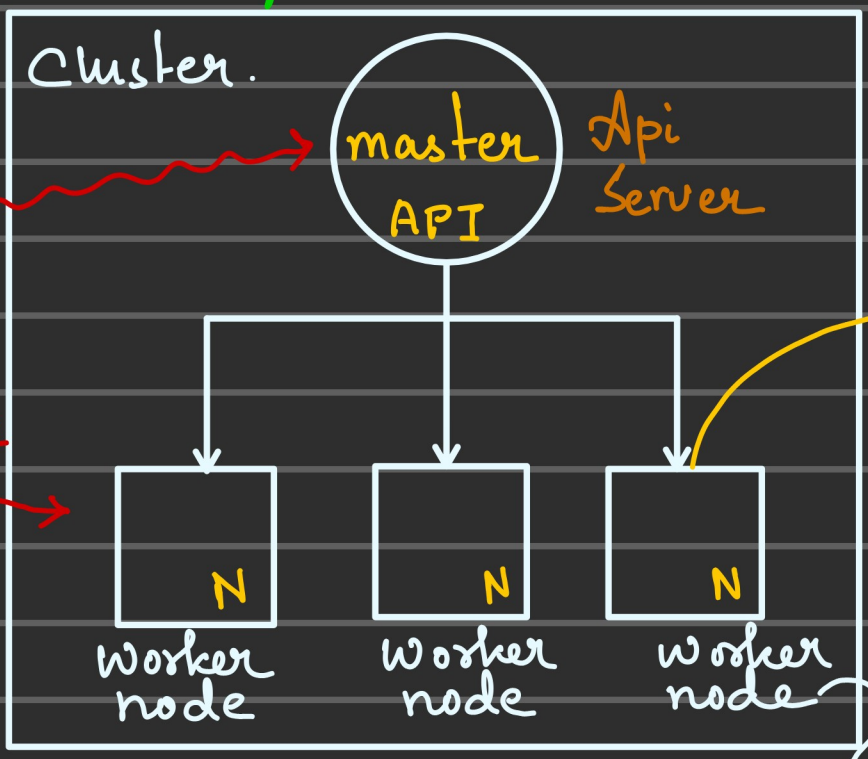
to manage a cluster
A Specialized node called Master

Notewise


Cluster
CONTROL PLANE
Cloud Controller Manager
Cloud provider API
ETCD
kube-API Server
Node 1
Node 2
kubelet
kube proxy
kubelet
kube proxy
Pod
Pod
Pod
Pod
CRI
CRI
Scheduler
Controller manager

Notewise

(Workflow diagram)



Components of Worker Node.

Kubelet: communicates with control plane (master)
→ Ensure all pods/ Containers are running.

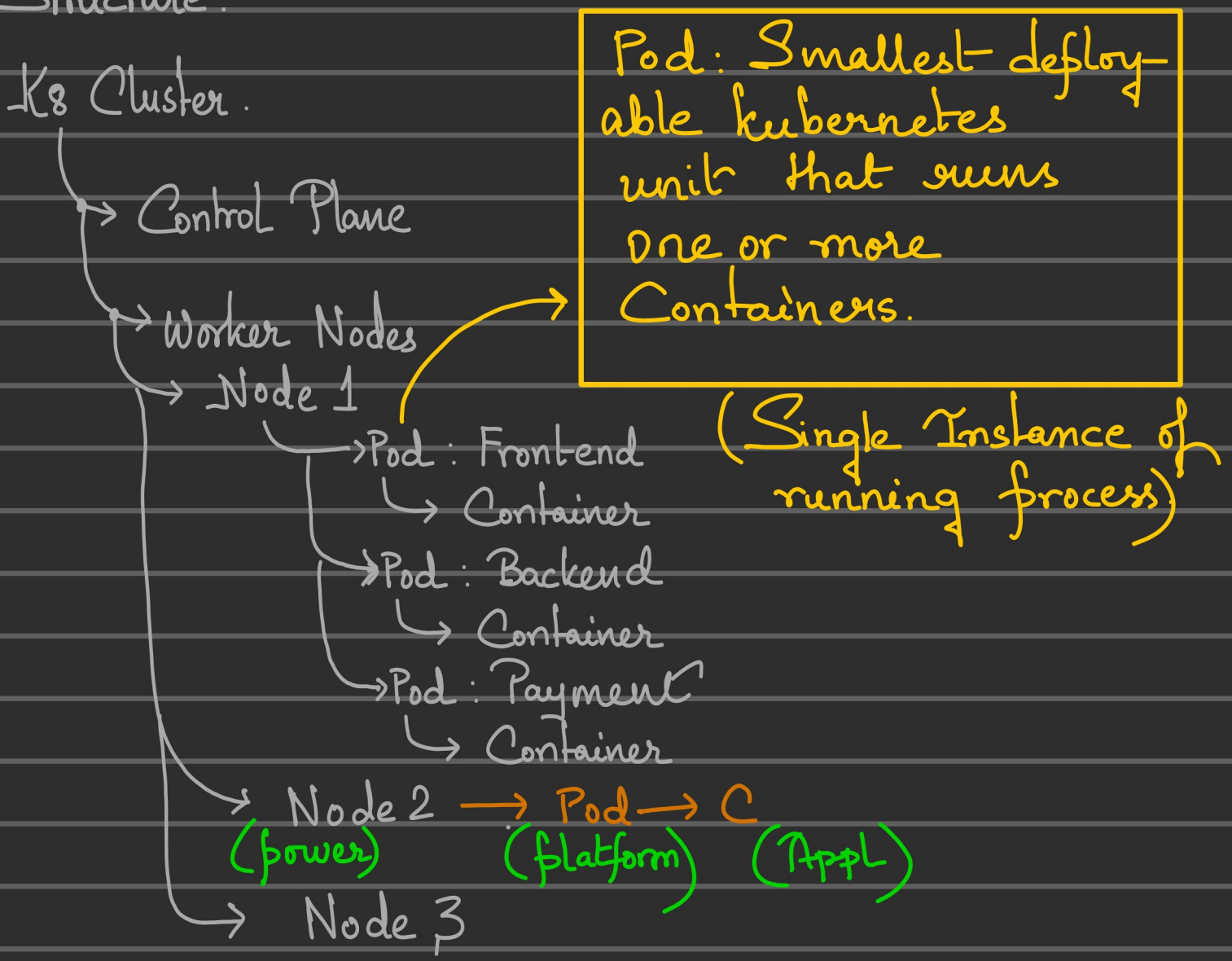
Container Runtime: Actually runs containers, eg containerd

kube-proxy: provides kubernetes network Service connectivity (Network rules).
→ Pod
(CNI)

Pods: Smallest deployable unit where Containers run.

Notewise

Structure:



Components of Master Node.

API Server: provides an Interface (kubectl) CLI to use K8.

Scheduler: Assign nodes to newly created Pods.

ETCD: Key-value store, having all cluster data, Config file, log file. state

Control Manager: responsible for managing services

Notewise

II.2.3 Self Healing & Managing Container health.

Example: Worker Node 1: Pod_1 → Container

Worker Node 2: Pod_2 (goes offline)

Auto deploy ⊕ ↘

Worker Node 3: Pod_3 + Pod_2

Self-Healing in Kubernetes.

feature: autodetects failed or unhealthy pods
↳ takes corrective actions.

mainly done thru controllers: Deployment & Replica Set: Compare Desired vs Current State.

Problem.	Action:
Container Crash	Restarts the container
Pod fails	Create replacement pod.
Node becomes unavailable	Reschedule Pods on healthy nodes.
Application becomes unhealthy	Restart appl/ replace pod.

Notewise

II.2.4 Replica Set, Deployment, Service

Replica Set: Ensures that a specified number of identical pods are running at all times

Deployment: Manages Replica Set and provides application updates and rollbacks

Handling Version Change:

Version 1 → nginx : 1.25
Version 2 → nginx : 1.26

D creates new replica set

gradually replace old pods

Service: provides stable network endpoint to access pod., can also load balance traffic

Workflow: Deployment

→ Replica set

[Pod 1, Pod 2, Pod 3]

→ Service ← User

Notewise